



**SmartLearn**

Connecter · Apprendre · Réussir

PROMPT DE GÉNÉRATION · CLAUDE CODE  
Compagnon du rapport CRM × SmartLearn

SCORING · DÉTECTION · RELANCE · DASHBOARD · MÉTRIQUE W4

# Moteur de Rétention Minimal

La seule brique CRM à construire pendant la Phase de Preuve  
Prompt prêt à coller dans Claude Code

RÉTENTION

W4 > 40 %

EDU

CAMEROUN · 2026

À coller dans Claude Code ouvert dans le dépôt SmartLearn existant

**Salomon FOE**

FONDATEUR & CEO · SMARTLEARN

salomonfoe97@smartlearn-edu.org · smartlearn-edu.org  
+237 671 719 124 · +237 691 276 334

DOCUMENT

Moteur de rétention — prompt

DATE

21 May 2026

## CONTEXTE

# La seule brique CRM de la Phase de Preuve

Ce document contient le **prompt de génération** du « moteur de rétention minimal » — extrait du rapport CRM × SmartLearn comme la **seule** brique CRM qui mérite ton temps pendant la Phase de Preuve. Il se branche sur SmartLearn edu existant (Next.js 16 + MongoDB Atlas) et sert directement le critère de sortie **W4 > 40 %**.

## ■ OPPORTUNITÉ

### Ce que ce module t'apporte

**(1)** La mesure réelle de ta rétention — tu sauras enfin si edu retient. **(2)** Le levier pour la faire monter — la relance automatique des décrocheurs. C'est exactement ce que la Phase de Preuve exige.

## ■ ATTENTION

### Avant de coller le prompt

Ouvre Claude Code **dans le dépôt SmartLearn existant**, pas dans un dossier vide. L'« Étape 0 — Inspection » est ce qui empêche le module de devenir un doublon parasite. Si Claude Code propose de recréer un modèle utilisateur ou une auth déjà présents, arrête-le et corrige.

## LE PROMPT

# À coller dans Claude Code

Colle l'intégralité des blocs ci-dessous (dans l'ordre) en une seule fois dans le chat Claude Code, puis envoie. Les sections sont séparées ici pour la lisibilité — c'est un seul et même prompt.

## ÉTAPE 0 — INSPECTION (obligatoire avant d'écrire du code)

Inspecte d'abord le code existant : modèles utilisateur/élève, cours, leçons, progression, paiements, connexion MongoDB, système d'auth, conventions de design. Réutilise-les. Ne crée aucun doublon. Si un système d'événements / analytics existe déjà, branche-toi dessus au lieu de créer learning\_event.

## CONTEXTE & RÔLE

Tu travailles À L'INTÉRIEUR du codebase existant de SmartLearn edu (Next.js 16 App Router, React 19, TypeScript, MongoDB Atlas, Tailwind, Vercel). Ta mission : ajouter un MOTEUR DE RÉTENTION MINIMAL comme module add-on. Tu ne reconstruis PAS edu. Tu ne touches pas à la logique cours / auth / paiement existante — tu la LIS seulement.

## OBJECTIF UNIQUE

Mesurer et améliorer la rétention des élèves. Ce module sert directement le critère de sortie de la Phase de Preuve : rétention hebdomadaire W4 > 40 %.

## PÉRIMÈTRE AUTORISÉ (et strictement rien d'autre)

1. Ingestion d'événements d'apprentissage
2. Score de santé élève (0-100) + 4 statuts
3. Détection automatique des décrocheurs
4. UNE séquence de ré-engagement (3 étapes)
5. Dashboard admin /admin/retention
6. Métrique de rétention W4 par cohorte hebdomadaire

## PÉRIMÈTRE INTERDIT (gel de scope — ne génère RIEN de ceci)

- Pas de builder de séquences multiples (UNE séquence, en dur)
- Pas de SMS/WhatsApp maintenant (interface abstraite prête, mais implémentation email/log uniquement)
- Pas de vues parent ni enseignant
- Pas de campagnes marketing, pas de pipeline commercial
- Pas de segmentation UI au-delà des 4 statuts
- Pas de refonte du design ni des pages existantes

## STACK & CONVENTIONS

- Next.js 16 App Router, Server Components par défaut, "use client" seulement pour l'interactivité
- MongoDB Atlas via le client existant du repo (lib/db ou équiv.)
- TypeScript strict, zod pour valider les entrées
- Tailwind, cohérent avec le design system existant
- Respecte tenant\_id sur chaque requête si le multi-tenant existe
- Vercel Cron pour les jobs planifiés

## MODÈLE DE DONNÉES (nouvelles collections, en plus de l'existant)

```
learning_event { _id, userId, tenantId, type, courseId?,
  lessonId?, value?, occurredAt }
type ∈ login | lesson_started | lesson_completed |
  quiz_submitted | payment_succeeded
(Si une collection d'events existe déjà, RÉUTILISE-LA.)
student_health { _id, userId, tenantId, score, status,
  recencyDays, activeDays14, lessonsCompleted14, lastActivityAt,
  computedAt }
status ∈ healthy | cooling | at_risk | churned
reengagement { _id, userId, tenantId, sequenceStep (1-3),
  status, triggeredAt, lastStepAt, recoveredAt? }
status ∈ active | recovered | completed | stopped
reengagement_log { _id, userId, step, channel, subject, body,
  sentAt, provider, delivered }
```

## ALGORITHME DE SCORING (recalculé chaque jour, idempotent)

```
score = 0.40*recency + 0.30*frequency + 0.30*progress (borné 0-100)
- recency : 100 si activité aujourd'hui, décroissance linéaire
  jusqu'à 0 sur 21 jours d'inactivité
- frequency : (jours actifs sur 14 derniers / 7) * 100, max 100
- progress : (leçons complétées sur 14 derniers / objectif) * 100
  objectif par défaut = 4 leçons / 14 jours
STATUTS :
- healthy : score >= 70
- cooling : 40-69
- at_risk : 15-39 OU inactivité entre 7 et 21 jours
- churned : score < 15 OU inactivité > 21 jours
```

## DÉTECTION & SÉQUENCE DE RÉ-ENGAGEMENT

- Cron quotidien : recalcule tous les student\_health.
- Quand un élève passe en at\_risk SANS reengagement actif → créer un reengagement (étape 1) et envoyer le message.
- Séquence en 3 étapes :
  - Étape 1 (J0) : « On t'a gardé ta place – reprends là où tu t'es arrêté » + lien dernière leçon
  - Étape 2 (J+3 si toujours inactif) : un conseil / ce qu'il rate
  - Étape 3 (J+7 si toujours inactif) : dernière relance + encouragement
- Si l'élève redevient actif à tout moment → status='recovered', recoveredAt=maintenant, sortie de séquence.
- Après l'étape 3 sans retour → status='completed' (décrochage confirmé).

## CANAL DE NOTIFICATION (abstrait — pour ajouter le SMS plus tard)

```
Crée une fonction sendReengagement(userId, step) qui :
- rend le template avec {{firstName}}, {{lastCourse}},
  {{lastLessonUrl}}
- envoie via Resend si RESEND_API_KEY défini, sinon LOG SEULEMENT
- écrit TOUJOURS un reengagement_log (testable sans provider)
Conçois l'interface pour qu'un canal 'sms' (Africa's Talking)
puisse être branché plus tard SANS réécrire la séquence.
```

## ROUTES API (app/api/)

```
POST /api/retention/events      → ingère un learning_event (zod)
GET  /api/retention/students    → liste élèves + santé (filtres)
GET  /api/retention/cohorts     → rétention W1 & W4 (6 cohortes)
POST /api/retention/reengage/[id] → déclenche une relance manuelle
GET  /api/cron/retention        → Vercel Cron (protégé
  CRON_SECRET) : recalcule scores + avance les séquences
```

## INSTRUMENTATION (brancher l'ingestion sur l'existant)

Ajoute des appels d'ingestion (POST /api/retention/events ou une fonction serveur `trackEvent()`) aux points existants : connexion, début de leçon, fin de leçon, soumission de quiz, paiement réussi. Ne duplique pas un système d'events déjà présent – branche-toi dessus.

## MÉTRIQUE W4 (définition exacte à implémenter)

- Cohorte = élèves inscrits dans une même semaine calendaire.
- Actif W4 =  $\geq 1$  `learning_event` significatif (`lesson_started`, `lesson_completed` ou `quiz_submitted`) dans la fenêtre [J21, J28] après la date d'inscription.
- Rétention W4 = (actifs W4) / (taille de la cohorte).

## DASHBOARD /admin/retention (accès admin uniquement)

- 4 cartes KPI : élèves actifs W4 · % rétention W4 (cible 40 % + écart) · élèves à risque · élèves récupérés (30 j)
- Donut : distribution des 4 statuts de santé
- Table cohortes : 6 dernières cohortes hebdo (W1 % et W4 %)
- Liste à-risque : élève, dernière activité, score, étape de séquence, bouton « Relancer »
- Feed des dernières relances (depuis `reengagement_log`)

## CONTRAINTES FINALES

- Réutilise modèles / DB / auth / design existants. Zéro doublon.
- Tout le texte UI en français.
- Aucune dépendance hors de ce que SmartLearn utilise déjà, + zod, + (optionnel) resend.
- Le recompute doit être idempotent.
- Données potentiellement de mineurs : ne logge jamais le contenu pédagogique sensible, seulement les métadonnées d'événement.

## À LA FIN, donne-moi

1. La liste exacte des fichiers créés et modifiés
2. La config Vercel Cron (`vercel.json`) du job quotidien
3. Les variables d'env (`CRON_SECRET`, `RESEND_API_KEY` optionnel)
4. Les points d'instrumentation ajoutés dans le code existant
5. La commande pour tester le recompute en local

## MODE D'EMPLOI

# Comment l'utiliser

- **Ouvre Claude Code dans le dépôt SmartLearn existant**, pas dans un dossier vide. L'étape 0 (inspection) empêche le doublon.
- **Vérifie l'étape 0 avant de laisser générer** : s'il propose de recréer un modèle ou une auth déjà présents, corrige (« réutilise le modèle X existant »).
- **Sans clé Resend**, le moteur *log* les relances dans le dashboard — tu peux donc le tester immédiatement, voir qui serait relancé, et brancher l'envoi réel (ou le SMS) plus tard.
- **Si la passe unique échoue**, itère par bouts : « refais seulement le scoring », « refais seulement le dashboard ».
- **Si tu n'as pas encore d'instrumentation analytics**, ce module la pose sur les 5 événements minimaux — cohérent avec le sprint S1-2 « Voir clair » du plan d'exécution.

## LE GARDE-FOU

## La discipline, encore

Ce module est la **seule chose CRM** que tu construis pendant la Phase de Preuve. Quand Claude Code — ou ton enthousiasme — proposera d'ajouter « tant qu'on y est » des campagnes, des segments, des vues parent, une vraie séquence multi-canal : **tu refuses**.

## ■ DÉCISION

**La règle d'arbitrage**

« *Est-ce que ça rapproche le critère W4 > 40 % ?* » Le scoring + la détection + une relance : **oui**. Le reste : **non** — c'est de la roadmap post-preuve.

FIN DU DOCUMENT

# SmartLearn

Connecter · Apprendre · Réussir

**Salomon FOE**

Fondateur · CEO SmartLearn

salomonfoe97@smartlearn-edu.org

+237 671 719 124 · +237 691 276 334

smartlearn-edu.org

Document généré le 21 May 2026 · Moteur de rétention minimal · Compagnon du rapport CRM × SmartLearn · Confidentiel